

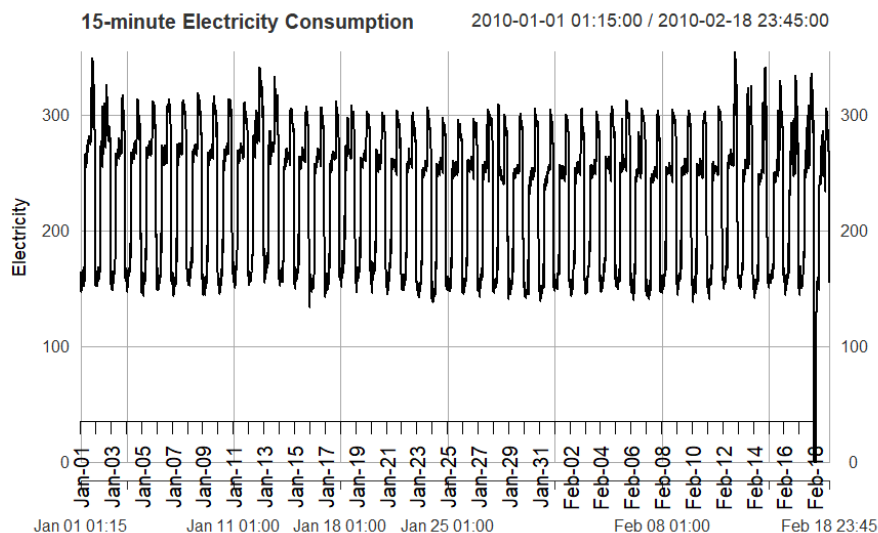
Electricity Consumption Forecasting

Introduction:

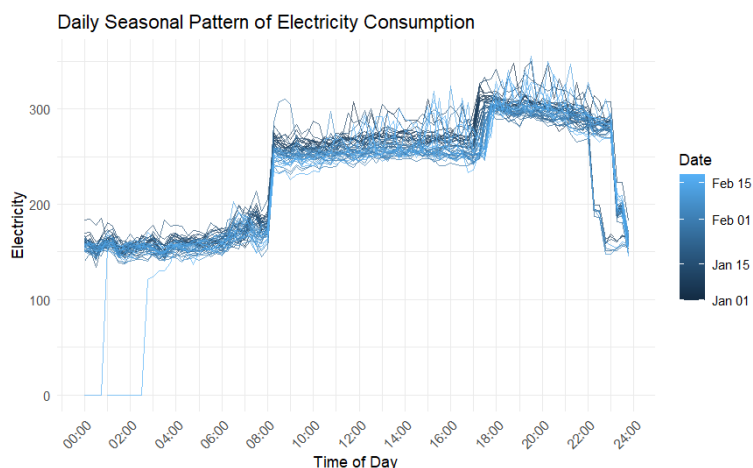
In this project, I work on electricity consumption data collected every 15 minutes from 1:15 of 1st January 2010 until 23:45 of 18th February 2010. There are two time series one is outdoor temperature and the other one is electricity consumption that we are interested in forecasting for the next day.

Data Analysis and Preparation

Firstly, I renamed the columned "Power (kW)" to "power", "Temp (C°)" to "temp" and "Timestamp" to "time" and extracted data from the excel file just until 23:45 of 18th February 2010. Then I plot the power as below:



I see that there seem to be daily seasonality in this series and I confirmed with the below seasonality plot.

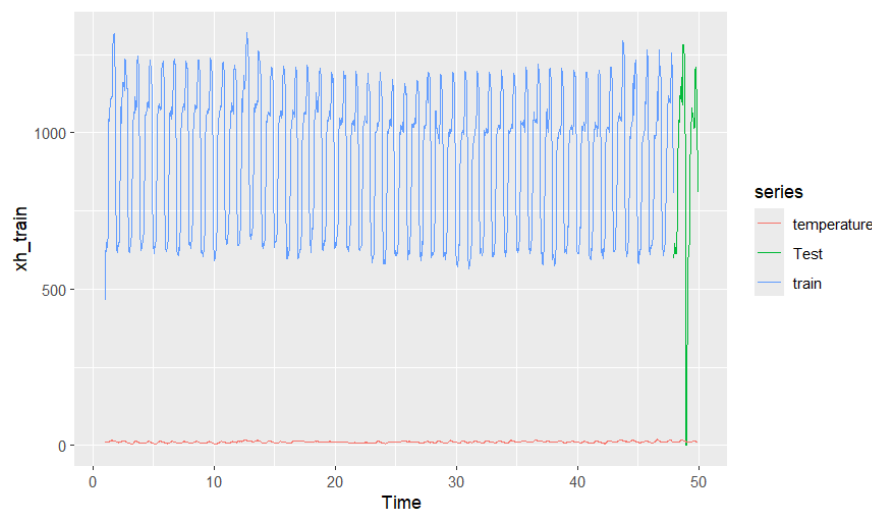


From the seasonality plot, we can see that there is daily pattern in this data set:

- From midnight until 8 am, the power consumption is around 150 to 200
- From 8 until 18, the power consumption is about 250 to 300
- From 18 until 23, the consumption is around 300 to 350.

However, there are some drops at the beginning of 18th Feb. so I decided to take the last two days as testing set so that we don't see a big error due to these drops and when I perform forecast for the next days by using the whole dataset, I imputed those drops so that the forecast can be more accurate.

Secondly, I aggregated the data into hourly data by summing the electricity power and averaging the temperature group by day. I did this in order to make the all forecasting models work and make them run faster and this is also because the function **hw** doesn't work with the frequency of 96. After that I took the last 48 hours as testing set and the remaining as training set as below plot.



Models Comparison

In order to compare all the forecasting models, I performed cross validation by using the training set. I take a window of 24 observations as validation fold and all the previous observations before those 24 observations as training fold. The window started from the first observation of the 5th day, which is at 0:00 so the first training fold is the first 4 days of the training set which are from first day at 0:00 until the 4th day at 23:00. Each iteration the window moves by one day, the forecasting models are fitted using the training fold, forecast the data the testing fold and I stored the computed mean absolute percentage error between the forecasts and the testing fold. After the cross validation I computed the mean and standard deviation of mean absolute percentage error of each folds.

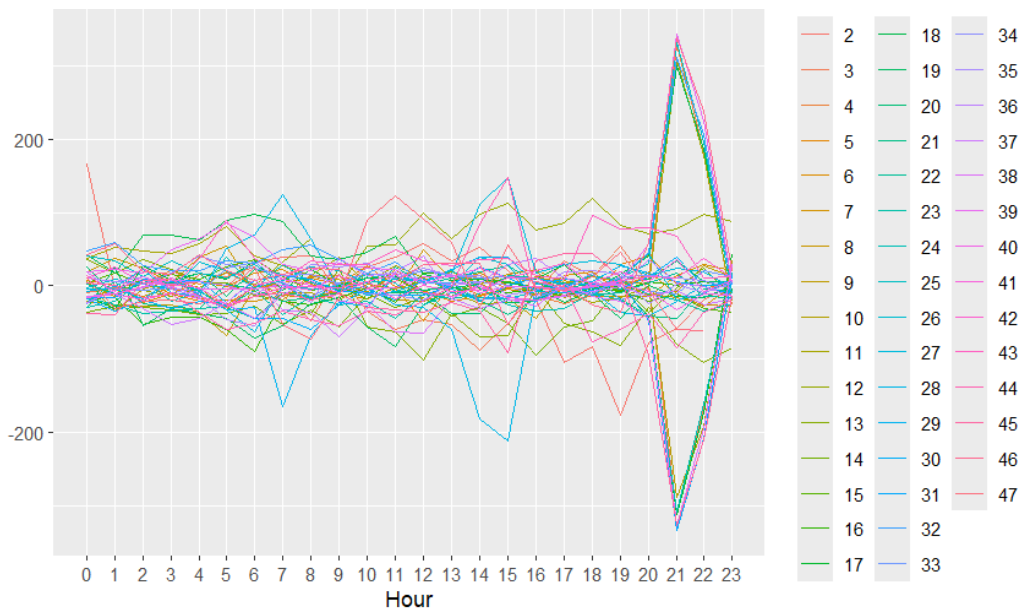
Exponential Smoothing Models

Firstly, I compare simple exponential smoothing, non-seasonal exponential smoothing, additive seasonal and multiplicative exponential smoothing and all those models with damped and as the result the multiplicative holt winter model has the lowest average of mean absolute percentage error as we can see below:

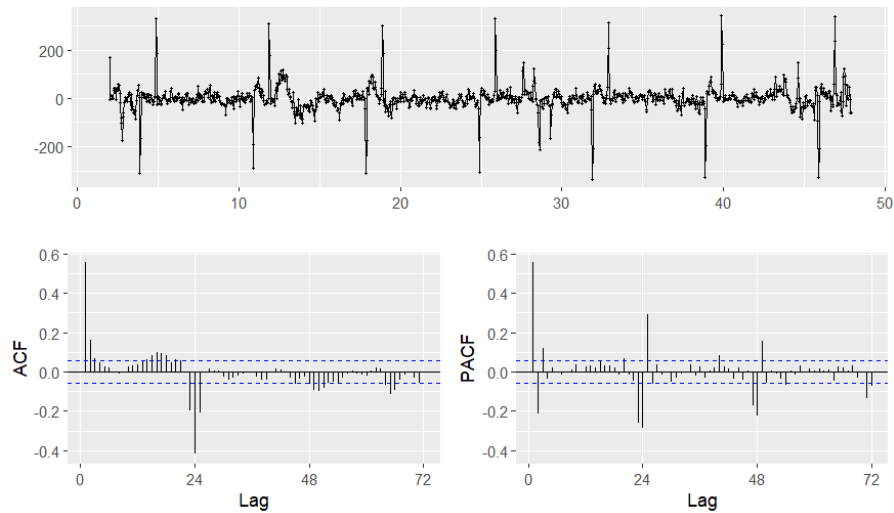
	Model	MAPE.Mean_MAPE	MAPE.SD_MAPE
1	add_seasonal	6.988593	7.509276
2	additive_seasonal_damped	7.406197	5.579099
3	multi_seasonal	6.645806	7.060471
4	multi_seasonal_damped	7.636998	5.885287
5	non_seasonal	227.148510	21.759818
6	non_seasonal_damped	72.738838	9.290462
7	SES	25.277141	1.541272

ARIMA Models

As there is daily seasonality in the data, I performed one difference operation on the data with lag 24 in order to make it stationary. From below seasonal plot of the diff(data, lag=24), we can conclude that there is no more clear seasonal patterns.



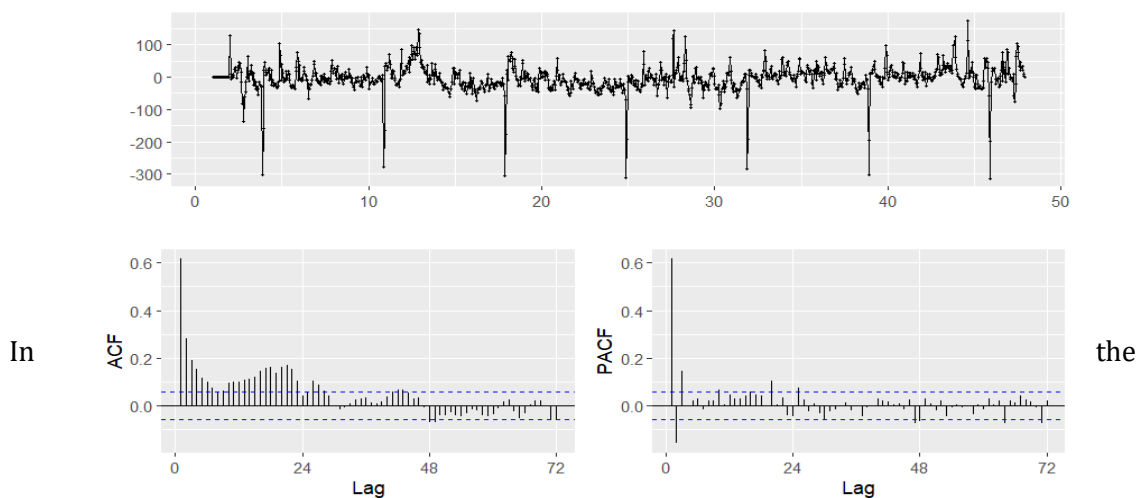
Next, I check the residual correlation in the below plot and I see that in PACF plot there is exponential decrease of the partial autocorrelation every season and in ACF plot, after the first season there is no significant autocorrelation. So, I tried **ARIMA(0,0,0)(0,1,1)24**



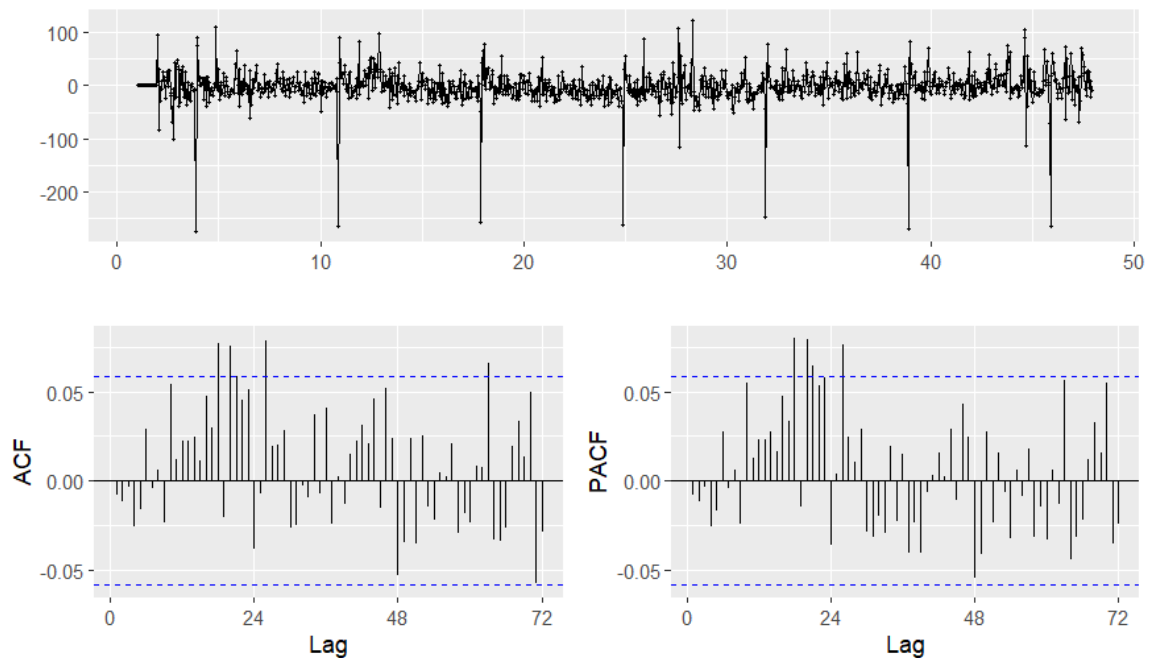
Then I performed cross validation with this model, and the result average MAPE is lower than all the Exponential smoothing models.

	Model	MAPE	Mean_MAPE	MAPE_SD_MAPE
1	add_seasonal	6.988593	7.509276	
2	additive_seasonal_damped	7.406197	5.579099	
3	arima(0,0,0)(0,1,1)24	3.007804	1.361609	
4	multi_seasonal	6.645806	7.060471	
5	multi_seasonal_damped	7.636998	5.885287	
6	non_seasonal	227.148510	21.759818	
7	non_seasonal_damped	72.738838	9.290462	
8	SES	25.277141	1.541272	

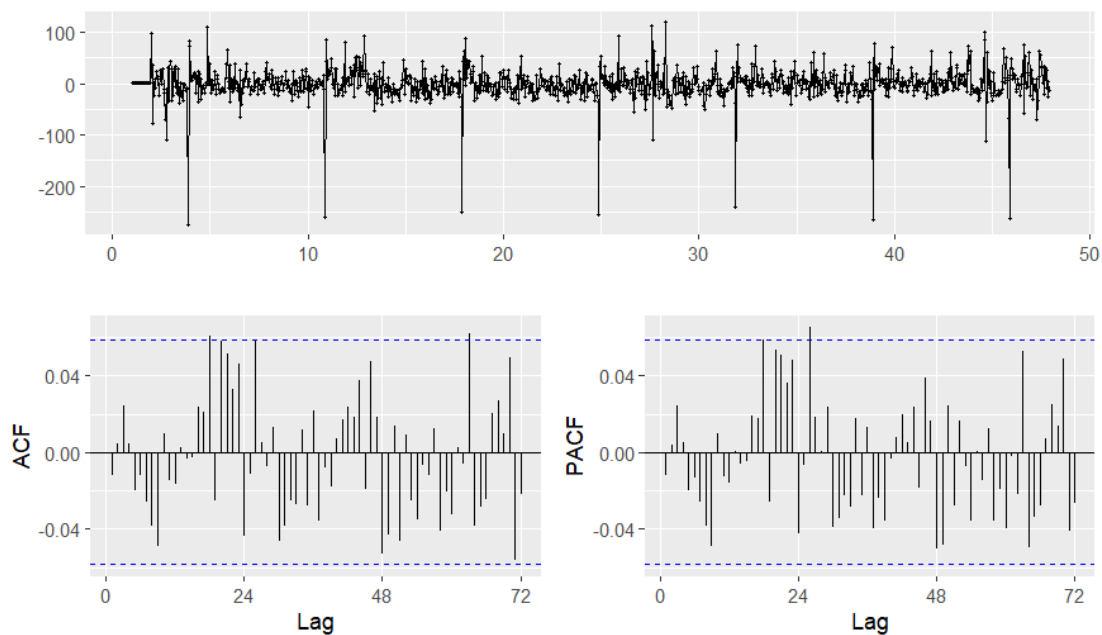
However, when I look at the residuals of this model, there are still correlations as below:



In ACF plot of the residuals we can see the decrease of autocorrelation and in the PACF we see that after the lag 3 there is no more significant autocorrelations, so I tried autoregressive of order 3: **Arima(3,0,0)(0,1,1)24**. However, there are still correlations as shown in the below plot:



Then I tried to increase the order of moving average to 3 so the model becomes **Arima(3,0,3)(0,1,1)24** and there is no more significant correlation in the residuals:



I came back to perform cross validation and as we can see in the below result, the **Arima(3,0,0)(0,1,1)** doesn't perform better than the previous one:

	Model	MAPE.Mean_MAPE	MAPE.SD_MAPE
1	add_seasonal	6.988593	7.509276
2	additive_seasonal_damped	7.406197	5.579099
3	arima(0,0,0)(0,1,1)24	3.007804	1.361609
4	arima(3,0,3)(0,1,1)24	3.452332	1.630175
5	multi_seasonal	6.645806	7.060471
6	multi_seasonal_damped	7.636998	5.885287
7	non_seasonal	227.148510	21.759818
8	non_seasonal_damped	72.738838	9.290462
9	SES	25.277141	1.541272

I tried also the model proposed by auto.arima function which is Arima(0,0,4)(1,1,0) and it does not perform better neither:

	Model	MAPE.Mean_MAPE	MAPE.SD_MAPE
1	add_seasonal	6.988593	7.509276
2	additive_seasonal_damped	7.406197	5.579099
3	arima(0,0,0)(0,1,1)24	3.007804	1.361609
4	arima(0,0,4)(1,1,0)24	3.276364	1.483146
5	arima(3,0,3)(0,1,1)24	3.452332	1.630175
6	multi_seasonal	6.645806	7.060471
7	multi_seasonal_damped	7.636998	5.885287
8	non_seasonal	227.148510	21.759818
9	non_seasonal_damped	72.738838	9.290462
10	SES	25.277141	1.541272

Neural Network Autoregressive

I turned the neural network model by searching for the best combination of autoregressive lag p, seasonal lag P and the number of neurons k that fit with the training set. I found the best parameters are p=25, P=0 and k=3.

Description: df [10 x 5]					
	p_small <dbl>	P <dbl>	k <dbl>	MAPE <dbl>	MAE <dbl>
49	25	0	3	2.669293	26.07178
53	25	1	3	2.669293	26.07178
52	25	0	10	2.754332	27.65316
56	25	1	10	2.754332	27.65316
51	25	0	7	2.765759	27.62478
55	25	1	7	2.765759	27.62478
45	20	2	3	2.788789	27.84361
43	20	1	7	2.853497	28.76175
47	20	2	7	2.888758	28.59539
46	20	2	5	2.914567	28.78139

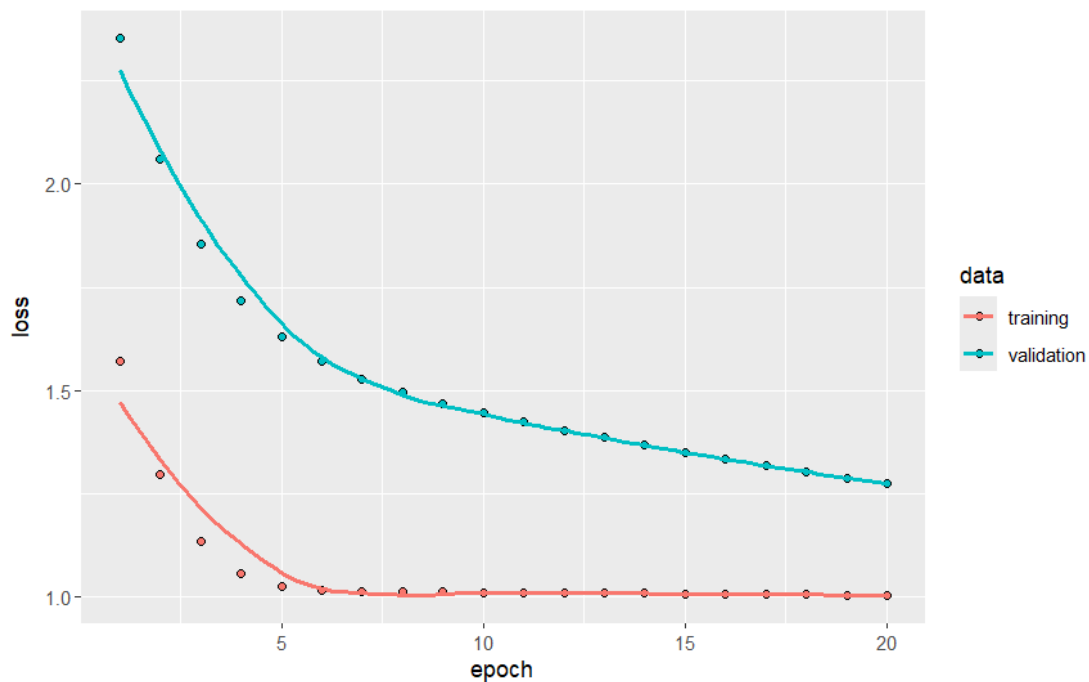
1-10 of 10 rows

With those parameters, I came back to perform cross validation to compare the model with the best parameters I found, and the neural network model proposed by automatic by nnetar function with the previous models. As we can see from the below result, the tuned neural network is slightly better than the model proposed by the nnetar function but still not better than Arima(0,0,0)(0,1,1).

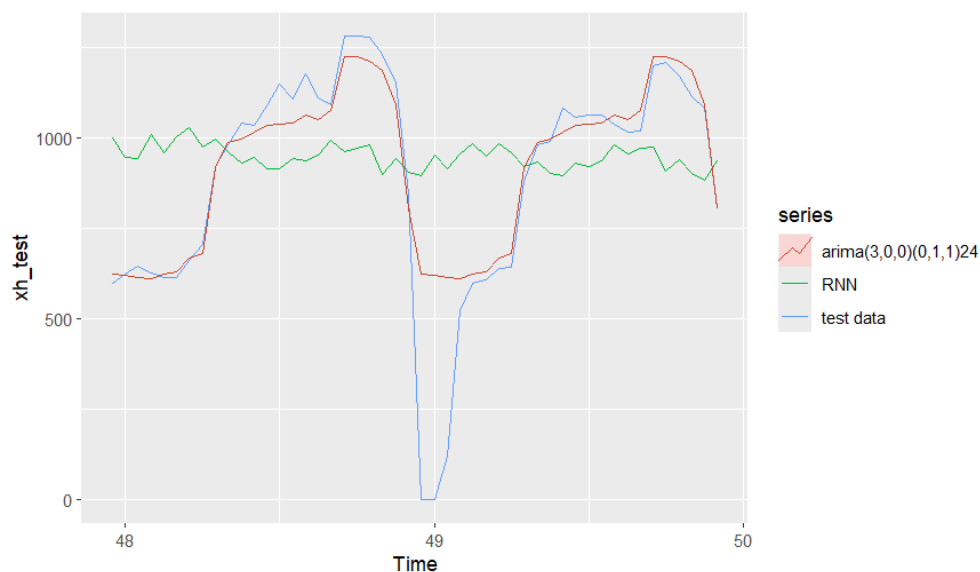
	Model	MAPE . Mean_MAPE	MAPE . SD_MAPE
1	add_seasonal	6.988593	7.509276
2	additive_seasonal_damped	7.406197	5.579099
3	arima(0,0,0)(0,1,1)24	3.007804	1.361609
4	arima(0,0,4)(1,1,0)24	3.276364	1.483146
5	arima(3,0,3)(0,1,1)24	3.452332	1.630175
6	multi_seasonal	6.645806	7.060471
7	multi_seasonal_damped	7.636998	5.885287
8	nn(25,0,3)	3.363547	1.936852
9	nn(25,1,13)	3.493808	2.109419
10	non_seasonal	227.148510	21.759818
11	non_seasonal_damped	72.738838	9.290462
12	SES	25.277141	1.541272

Recurrent Neural Network

I tried to tune RNN manually by looking at the validation curve to make sure that the model learning without becoming overfitted. I chose the window of 24 observations as input to train the model with 10 neurons, batch size of 64 and 20 epochs.



Then I checked the forecast of RNN for the testing dataset and Arima MAE is 70.01404 while RNN MAE is 257.0453 which is much worse. The plot of the forecast below we clearly see that RNN can't be used to forecast the future of our time series.



Random Forest

I tuned random forest model with the training set by searching for the best combination of number variable to be split **mtry** and size of the input **window**. The best parameters I found are **mtry = 25** and **window = 30**.

Description: df [20 x 4]				
	window <dbl>	mtry <dbl>	MAE <dbl>	MAPE <dbl>
30	30	25	22.13900	2.436808
23	28	25	22.23518	2.441339
22	28	20	22.33139	2.451306
38	40	30	22.59655	2.481425
29	30	20	22.58647	2.487521
28	30	15	22.78949	2.519663
37	40	25	22.90536	2.526240
21	28	15	23.05239	2.547273
36	40	20	23.19492	2.567789
20	28	10	23.41914	2.593419

1-10 of 20 rows

With those parameters I come back to cross validation and we can see from the below result that it is better than other models but still can't beat the Arima(0,0,0)(0,1,1).

	Model	MAPE	Mean_MAPE	MAPE_SD_MAPE
1	add_seasonal	6.988593	7.509276	
2	additive_seasonal_damped	7.406197	5.579099	
3	arima(0,0,0)(0,1,1)24	3.007804	1.361609	
4	arima(0,0,4)(1,1,0)24	3.276364	1.483146	
5	arima(3,0,3)(0,1,1)24	3.452332	1.630175	
6	multi_seasonal	6.645806	7.060471	
7	multi_seasonal_damped	7.636998	5.885287	
8	nn(25,0,3)	3.363547	1.936852	
9	nn(25,1,13)	3.493808	2.109419	
10	non_seasonal	227.148510	21.759818	
11	non_seasonal_damped	72.738838	9.290462	
12	random forest	3.106068	1.381782	
13	SES	25.277141	1.541272	

XGBoost

Similarly, I tuned XGboost model by searching for the best combination of the maximum depth, learning rate, and input window size. The best parameters are window = 30, max_depth = 8 and learning rate = 0.05.

Description: df [20 x 5]					
	window <dbl>	max_depth <dbl>	learning_rate <dbl>	MAE <dbl>	MAPE <dbl>
58	30	8	0.05	20.20282	2.226694
59	30	8	0.10	20.51566	2.247259
79	40	10	0.10	20.15854	2.250331
50	30	3	0.05	20.59574	2.274922
54	30	5	0.05	20.68888	2.276466
72	40	5	0.30	20.52234	2.278348
53	30	5	0.01	20.75754	2.278385
80	40	10	0.30	20.30364	2.279652
76	40	8	0.30	20.27575	2.283687
62	30	10	0.05	20.74576	2.285363

1-10 of 20 rows

Previous 1 2 Next

With cross validation result, we can see that XGboost doesn't perform the best.

	Model	MAPE.Mean_MAPE	MAPE.SD_MAPE
1	add_seasonal	6.988593	7.509276
2	additive_seasonal_damped	7.406197	5.579099
3	arma(0,0,0)(0,1,1)24	3.007804	1.361609
4	arma(0,0,4)(1,1,0)24	3.276364	1.483146
5	arma(3,0,3)(0,1,1)24	3.452332	1.630175
6	multi_seasonal	6.645806	7.060471
7	multi_seasonal_damped	7.636998	5.885287
8	nn(25,0,3)	3.363547	1.936852
9	nn(25,1,13)	3.493808	2.109419
10	non_seasonal	227.148510	21.759818
11	non_seasonal_damped	72.738838	9.290462
12	random forest	3.106068	1.381782
13	SES	25.277141	1.541272
14	xgboost	3.314964	1.816025

SVM

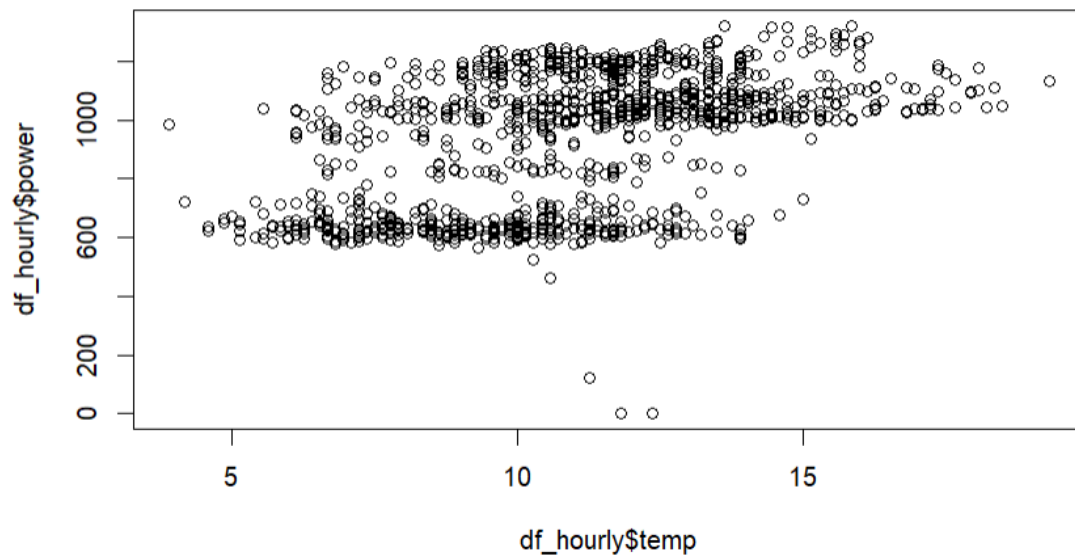
The same for SVM it doesn't win over ARIMA.

	Model	MAPE.Mean_MAPE	MAPE.SD_MAPE
1	add_seasonal	6.988593	7.509276
2	additive_seasonal_damped	7.406197	5.579099
3	arma(0,0,0)(0,1,1)24	3.007804	1.361609
4	arma(0,0,4)(1,1,0)24	3.276364	1.483146
5	arma(3,0,3)(0,1,1)24	3.452332	1.630175
6	multi_seasonal	6.645806	7.060471
7	multi_seasonal_damped	7.636998	5.885287
8	nn(25,0,3)	3.363547	1.936852
9	nn(25,1,13)	3.493808	2.109419
10	non_seasonal	227.148510	21.759818
11	non_seasonal_damped	72.738838	9.290462
12	random forest	3.106068	1.381782
13	SES	25.277141	1.541272
14	svm	3.392291	1.487018
15	xgboost	3.314964	1.816025

Forecast with covariate

Since we have seen many models and none of them performed better than ARIMA, next I tried to use temperature time series to help the forecast of electricity consumption. First, I tried to look if there is any relationship between the two series:

From this plot we don't see a strong relationship between the two series, but we can see



that when the temperature is more than 15 the consumption seems to be higher than 700. When the temperature is around 5 the power consumption is less than 1000

First I use lstm function to extract the residuals:

```

Call:
tslm(formula = xh_train ~ zh_train + season + trend)

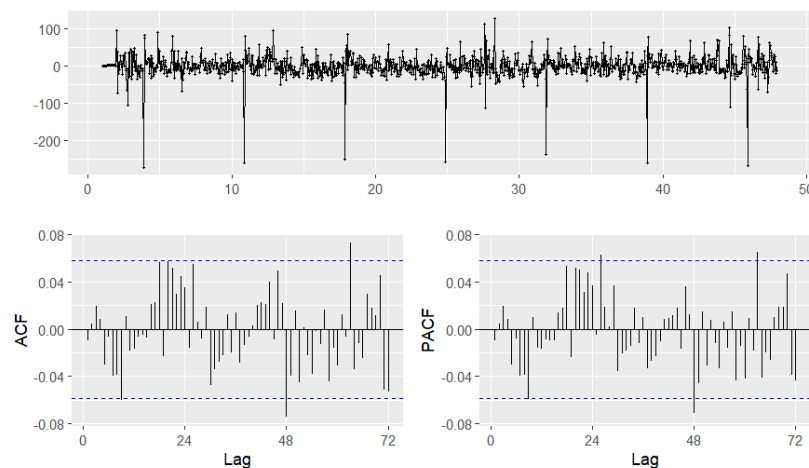
Residuals:
    Min       1Q   Median       3Q      Max
-305.312 -17.319   1.563  18.929 151.532

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 599.107867   8.483949  70.617 < 2e-16 ***
zh_train      5.414183   0.653248   8.288 3.32e-16 ***
season2       0.860622   8.573199   0.100  0.9201
season3      -1.120387   8.577475  -0.131  0.8961
season4      12.521405   8.586352   1.458  0.1450
season5      19.804158   8.593998   2.304  0.0214 *
season6      53.402004   8.596204   6.212 7.39e-10 ***
season7      85.184118   8.595588   9.910 < 2e-16 ***
season8     332.125487   8.590590  38.662 < 2e-16 ***
season9     401.410943   8.572336  46.826 < 2e-16 ***
season10    396.756943   8.602536  46.121 < 2e-16 ***
season11    403.768298   8.684466  46.493 < 2e-16 ***
season12    411.239547   8.802101  46.721 < 2e-16 ***
season13    412.143752   8.904653  46.284 < 2e-16 ***
season14    409.797213   9.005142  45.507 < 2e-16 ***
season15    412.499043   9.061363  45.523 < 2e-16 ***
season16    409.269265   9.005028  45.449 < 2e-16 ***
season17    494.159646   8.872337  55.697 < 2e-16 ***
season18    578.920627   8.739630  66.241 < 2e-16 ***
season19    581.612680   8.685870  66.961 < 2e-16 ***
season20    567.106878   8.648618  65.572 < 2e-16 ***
season21    548.309150   8.615758  63.640 < 2e-16 ***
season22    470.682406   8.593024  54.775 < 2e-16 ***
season23    183.517480   8.582688  21.382 < 2e-16 ***
season24     4.091502   8.620129   0.475  0.6351
trend       -0.055728   0.003998 -13.940 < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

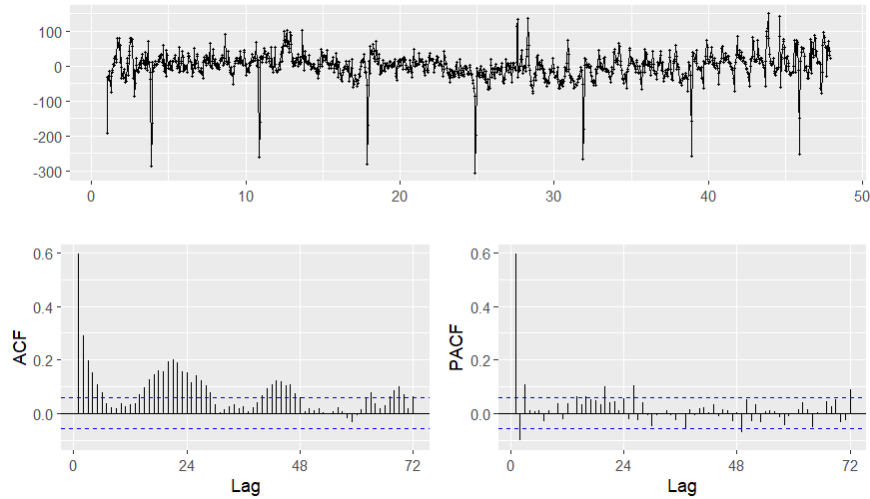
Residual standard error: 41.56 on 1101 degrees of freedom
Multiple R-squared:  0.9664,    Adjusted R-squared:  0.9657
F-statistic: 1268 on 25 and 1101 DF,  p-value: < 2.2e-16

```

From this tslm summary, we see that most of the coefficient are significant, and we have positive coefficient for the temperature and the trend is negative. In the residual plot below we can see that there is correlation and seasonality.



Similar to what I did previously for Arima model, I found the Arima(3,0,3)(0,1,2) can modeled all the correlation in the residuals



I performed cross validation on all the Arima I fitted with covariates that I found to compare with all the previous models I tried and as the result no model can win against Arima (0,0,0)(0,1,1)

	Model	MAPE	Mean_MAPE	MAPE_SD_MAPE
1	add_seasonal	6.988593	7.509276	
2	additive_seasonal_damped	7.406197	5.579099	
3	arima(0,0,0)(0,1,1)24	3.007804	1.361609	
4	arima(0,0,4)(1,1,0)24	3.276364	1.483146	
5	arima(3,0,3)(0,1,1)24	3.452332	1.630175	
6	arima_covar(0,0,0)(0,1,1)24	3.304291	1.643840	
7	arima_covar(0,0,1)(1,1,0)24	3.406584	1.443530	
8	arima_covar(0,0,2)(1,1,0)24	3.433968	1.434184	
9	multi_seasonal	6.645806	7.060471	
10	multi_seasonal_damped	7.636998	5.885287	
11	nn(25,0,3)	3.386105	1.969563	
12	nn(25,1,13)	3.569337	2.146321	
13	non_seasonal	227.148510	21.759818	
14	non_seasonal_damped	72.738838	9.290462	
15	random forest	3.106068	1.381782	
16	SES	25.277141	1.541272	
17	xgboost	3.314964	1.816025	

The same for Neural network, Random Forest, and XGBoost, the covariate doesn't help them to perform better

	Model	MAPE . Mean_MAPE	MAPE . SD_MAPE
1	add_seasonal	6.988593	7.509276
2	additive_seasonal_damped	7.406197	5.579099
3	arima(0,0,0)(0,1,1)24	3.007804	1.361609
4	arima(0,0,4)(1,1,0)24	3.276364	1.483146
5	arima(3,0,3)(0,1,1)24	3.452332	1.630175
6	arima_covar(0,0,0)(0,1,1)24	3.304291	1.643840
7	arima_covar(0,0,1)(1,1,0)24	3.406584	1.443530
8	arima_covar(0,0,2)(1,1,0)24	3.433968	1.434184
9	multi_seasonal	6.645806	7.060471
10	multi_seasonal_damped	7.636998	5.885287
11	nn(25,0,3)	3.386105	1.969563
12	nn(25,1,13)	3.569337	2.146321
13	nn_covar(25,0,3)	3.619103	2.022065
14	non_seasonal	227.148510	21.759818
15	non_seasonal_damped	72.738838	9.290462
16	random forest	11.382255	11.822779
17	random forest with covariate	21.227905	9.592318
18	SES	25.277141	1.541272
19	xgboost	3.314964	1.816025
20	XGBoost with covariate	7.873804	1.289414

Weighted Near Neighbor

After implementing the WNN model, I tuned the parameters with hourly data by performing cross validation with different combinations of window, and k. I found that the best parameters are window = 35 and k = 5.

	w <dbl>	k <dbl>	MAE <dbl>	SD <dbl>	Folds <int>
38	35	5	26.26495	10.49316	43
20	24	5	26.44302	12.12231	43
44	40	5	26.72156	10.56546	43
19	24	3	26.74316	12.71746	43
26	28	5	26.75404	11.22372	43
32	30	5	26.84911	11.27248	43
21	24	8	27.17853	11.87576	43
39	35	8	27.18274	11.08820	43
43	40	3	27.18849	11.81950	43
33	30	8	27.35719	11.40785	43

1-10 of 54 rows

Previous 1 2 3 4 5 6 Next

Then I come back to check cross validation errors with all the previous models and as the below result, WNN performs the best among all the models so far.

	Model	MAPE . Mean_MAPE	MAPE . SD_MAPE
1	add_seasonal	6.988593	7.509276
2	additive_seasonal_damped	7.406197	5.579099
3	arima(0,0,0)(0,1,1)24	3.007804	1.361609
4	arima(0,0,4)(1,1,0)24	3.276364	1.483146
5	arima(3,0,3)(0,1,1)24	3.452332	1.630175
6	multi_seasonal	6.645806	7.060471
7	multi_seasonal_damped	7.636998	5.885287
8	nn(25,0,3)	3.363547	1.936852
9	nn(25,1,13)	3.493808	2.109419
10	nn_covar(25,0,3)	3.619103	2.022065
11	non_seasonal	227.148510	21.759818
12	non_seasonal_damped	72.738838	9.290462
13	random forest	3.106068	1.381782
14	SES	25.277141	1.541272
15	svm	3.392291	1.487018
16	WNN	2.971335	1.136200
17	xgboost	3.314964	1.816025

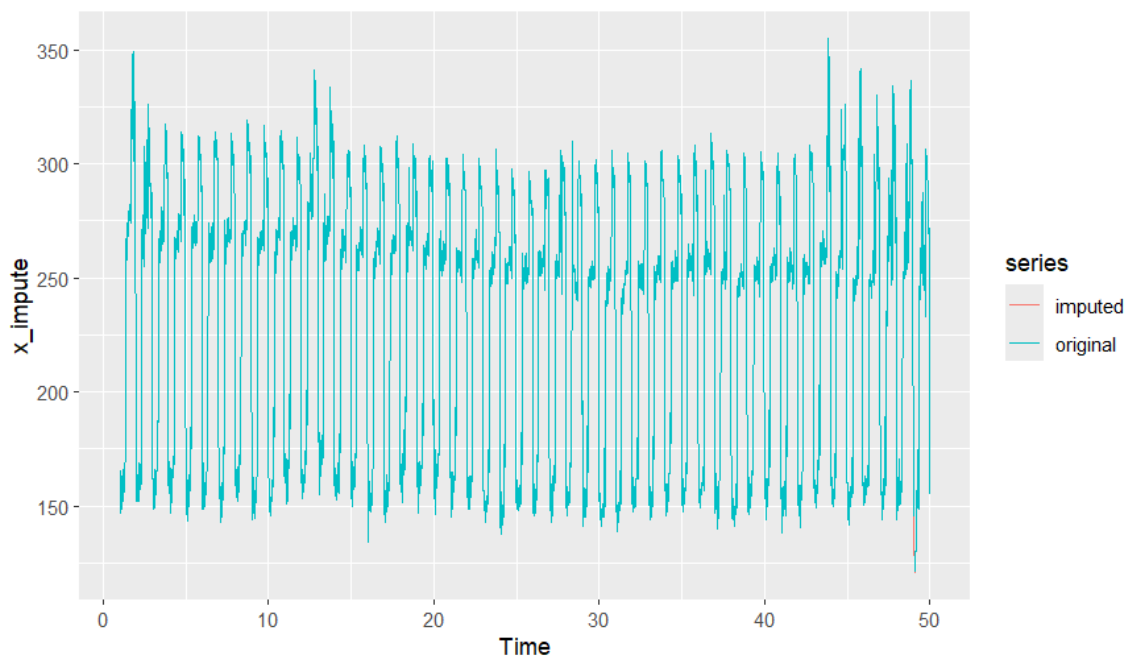
However, when I fitted those model, whose errors in average are small around 3 on cross validation, with the full training data and forecast on the test set and then computed test error. It turned out that Arima model with covariate and order (0,0,2)(1,10) performed the best. However, it is just on one test set.

Therefore, I decided to select the Arima(0,0,0)(0,1,1) to forecast the future value because it is the most stable one among others even in cross validation WNN performed slightly better it but on test set the Arima(0,0,0)(0,1,1) is better than WNN. While the Arima with covariate (0,0,2)(1,1,0) performed the best on test set but it could be the best just only on one test set because we saw in cross validation it didn't perform the best.

Description: df [10 × 2]

Model <chr>	MAE <dbl>
Arima(0,0,0)(0,1,1)	69.44289
Arima(3,0,3)(0,1,1)	70.15203
Arima(0,0,4)(1,1,0)	69.87837
nn	68.30005
Random Forest	69.13154
XgBoost	71.30657
Arima_covariate	70.35882
Arima_covariate(0,0,0)(0,1,1)	68.09920
Arima_covariate(0,0,2)(1,1,0)	66.43073
WNN	81.47532

Finally, I came back to the original data with frequency = 96. I imputed the data whose value = 0 before fitting all the available data to forecast the next future day. I used the function **na_seadec** and algorithm is **interpolation**.



Here is the final forecast:

